🕐

Date.prototype.getHours()

📄 Título

`Date.prototype.getHours()` en JavaScript

📝 Definición

- `getHours()` es un **método** del objeto `Date` que devuelve la **hora** de una fecha según la **hora local**, representada mediante un número entre **0 y 23**.

> **Importante:** Utiliza el formato de **24 horas**, no el de 12 horas.

🧠 ¿Qué es?

- Es un método que permite obtener la hora de una fecha.
- Devuelve un valor de tipo **number**.
- Utiliza la zona horaria local del sistema.

🎯 ¿Para qué sirve?

Se utiliza para:

- Mostrar la hora actual.
- Crear relojes digitales.
- Validar horarios.
- Programar tareas según la hora.
- Formatear fechas y horas.

⚙️ ¿Cómo funciona?

1. Se crea un objeto `Date`.
2. Se llama al método `getHours()`.
3. JavaScript obtiene la hora local almacenada en el objeto.
4. Devuelve un número entre **0 y 23**.

Ejemplo del proceso:

```
const fecha=new Date("2026-07-22T18:45:30");console.log(fecha.getHours());
```

Resultado:

```
18
```

📝 Sintaxis / Estructura (JavaScript)

```
fecha.getHours()
```

Parámetros

- No recibe parámetros.

Valor de retorno

- Un **number** entre **0 y 23**.
- Devuelve `NaN` si la fecha es inválida.

💡 Ejemplos

Ejemplo 1: Hora actual

```
const ahora=new Date();
```

```
console.log(ahora.getHours());
```

Resultado (ejemplo):

```
17
```

Ejemplo 2: Hora de una fecha específica

```
const fecha=new Date("2026-07-22T09:30:00");
console.log(fecha.getHours());
```

Resultado:

```
9
```

Ejemplo 3: Medianoche

```
const fecha=new Date("2026-07-22T00:00:00");
console.log(fecha.getHours());
```

Resultado:

```
0
```

Ejemplo 4: Última hora del día

```
const fecha=new Date("2026-07-22T23:59:59");
console.log(fecha.getHours());
```

Resultado:

```
23
```

Ejemplo 5: Fecha inválida

```
const fecha=new Date("Fecha inválida");
console.log(fecha.getHours());
```

Resultado:

```
NaN
```

⚠️ Errores comunes

❌ Pensar que devuelve el formato de 12 horas

```
const fecha=new Date("2026-07-22T15:00:00");
console.log(fecha.getHours());
```

Resultado:

```
15
```

No devuelve:

```
3 PM
```

Devuelve la hora en formato de **24 horas**.

❌ Confundir `getHours()` con `getUTCHours()`

```
const fecha=new Date();
console.log(fecha.getHours());
```

`getHours()` utiliza la **hora local**.

Mientras que:

```
console.log(fecha.getUTCHours());
```

Devuelve la hora en **UTC (Tiempo Universal Coordinado)**.

❌ Llamarlo sobre una fecha inválida

```
const fecha=new Date("abc");
console.log(fecha.getHours());
```

Resultado:

```
NaN
```

💼 Casos de uso

- Mostrar la hora actual.
- Crear relojes digitales.
- Programar recordatorios.
- Validar horarios de atención.
- Registrar eventos con fecha y hora.

🧩 Relación con otros conceptos

- `getMinutes()` → Devuelve los minutos (0–59).
- `getSeconds()` → Devuelve los segundos (0–59).
- `getMilliseconds()` → Devuelve los milisegundos (0–999).
- `getUTCHours()` → Devuelve la hora en UTC.
- `Date` → Objeto que representa fechas y horas.

🧠 Analogía

Imagina un **reloj digital** 🕒.

Si el reloj muestra:

```
18:45:30
```

Cada método obtiene una parte diferente:

- 🕒 `getHours()` → **18**
- ⏱️ `getMinutes()` → **45**
- ⏲️ `getSeconds()` → **30**

`getHours()` solo se fija en la **hora**, ignorando los minutos y los segundos.

📚 Fuente

- Documentación oficial de JavaScript (MDN).
- Especificación ECMAScript.

📝 Notas Personales

- ✅ `getHours()` devuelve la **hora local**.
- ✅ El rango va de **0 a 23**.
- ✅ Devuelve un valor de tipo **number**.
- ✅ No recibe parámetros.
- ✅ Utiliza el formato de **24 horas**.
- ❌ No debe confundirse con `getUTCHours()`, que utiliza la hora UTC.